

Sorting and Patterns - Day 3

Merge, partition, and shrinking windows

Priyanka's Platinum study plan | Wednesday, September 9, 2026 | 120 minutes | C11 and Python 3

Today's question: how do local region promises compose into a correct global answer? Prerequisites: Day 1's sorted-region proofs, Day 2's aggregate analysis, and basic recursion. Begin in Python and translate to C. Reuse your test harness. Do not read the solution appendix until you attempt the relevant lab.

Your two-hour session

Minutes	Task	Deliverable
0-15	Trace merge and partition	Two region diagrams
15-45	Lab A: merge sort in both languages	Reusable buffer, correct endpoints
45-75	Lab B: quicksort in both languages	Partition plus terminating recursion
75-90	Solve recurrence problems	Balanced and unbalanced costs
90-115	Lab C: variable window in both languages	Repeated shrinking and edge tests
115-120	Exit and correction	Positivity counterexample

Lab A - Merge two promises

Motivation: a quadratic sort repeatedly revisits a growing portion of the array. If two halves are already sorted, can we combine them with one scan? First merge $[2,4,5]$ and $[1,2,3]$ by hand. At each step choose the smaller next unused value; tie-break to the left. This preserves the order of equal-key records across the two runs.

Use half-open intervals $[lo,hi)$. Empty and singleton intervals satisfy $hi-lo < 2$. Split at $mid = lo + (hi-lo)/2$. The recursive contract is: after `visit(lo,hi)`, that interval is a sorted permutation of its original contents and other positions are unchanged. Before merging, $[lo,mid)$ and $[mid,hi)$ are sorted.

Merge invariant: `buffer[lo:k]` is sorted and contains exactly the consumed elements from both runs. Each input pointer identifies its next unused element. The next smallest value must be one of those two heads. Choosing it preserves order and content. At $k=hi$ all elements are consumed, so copy the interval back to `a`. Keep reading from `a` and writing into the separate buffer until the merge is complete.

Implement one buffer allocated at the top level and reused by all recursive calls. C returns false if allocation fails and true on success; avoid multiplying `n` by `sizeof(int)` without checking overflow. Free the buffer once. Python builds one list of length `n`, not a new slice for each recursive call.

Trace $[5,2,4,2,1]$, including its uneven split. Tests: empty, singleton, sorted, reversed, all equal, and negative values. Tempting bug: merge directly into `a` while its right-hand run still contains unread data. Find a failing merge with $[2,4]$ and $[1,3]$.

Lab B - Partition before recursion

Quicksort asks a different question: can we place one pivot so all remaining sorting is confined to its sides? Use Lomuto partition with an INCLUSIVE `hi` and `pivot=a[hi]`. This convention is intentionally distinct from merge sort: name it before coding.

State before scan index `j`: $[lo,i)$ contains values $\leq pivot$, $[i,j)$ contains values $> pivot$, $[j,hi)$ is unprocessed, and pivot remains at `hi`. If `a[j] ≤ pivot`, swap it into `i` and advance `i`. In either branch, advance `j`. When `j=hi`, swap the pivot into `i` and return `i`. Every element left of it is $\leq pivot$ and every element right is $> pivot$.

Trace [4,2,5,2,3]. Write lo,i,j,hi and the array after each swap. Then recurse on [lo,p-1] and [p+1,hi], excluding the pivot. Prove both intervals are smaller. In C, do not form p-1 when p=0; guard p>lo. Return immediately for arrays of length below two before constructing n-1.

Tests: [2,1], [1,2], [2,2,2,2], sorted distinct, reverse-sorted, and empty. Explain why this partition is generally unstable. A production implementation needs more protection against bad partitions; this small reference deliberately exposes them. Keep adversarial quicksort tests small (for example ≤ 128) so Python's recursion limit or the C stack does not obscure the lesson.

Recurrence workshop

1. For merge sort, write a recurrence with uneven halves. For n a power of two, count work per level and the number of levels. Why does already sorted input still require full merge scans in this implementation?
2. Quicksort's partition is linear. Derive balanced and maximally unbalanced recurrences. On sorted input with the last pivot, which one occurs?
3. All-equal input also produces a partition of sizes n-1 and 0. Would merely choosing a random pivot fix that for this two-way partition?
4. Separate output array storage, merge buffer space, and recursive stack space. 'In-place partition' does not imply constant total quicksort space.

Lab C - The shortest valid positive window

Given a list of POSITIVE integers and positive target, return the minimum length of a contiguous subarray with $\text{sum} \geq \text{target}$, or 0 if none exists. An empty input returns 0. Example [2,3,1,2,4,3], target 7 $\rightarrow$ 2, using [4,3]. Do not sort: sorting changes contiguity. For the C exercises use $n \leq 1000000$, 32-bit int values and a long long sum/target.

The direct method examines every start and endpoint, taking $O(n^2)$ time. Maintain left and an exact running sum while extending right. Whenever $\text{sum} \geq \text{target}$, record the current length, remove $a[\text{left}]$, and advance left. Repeat while valid. For a fixed right endpoint, repeated shrinking finds its shortest currently available valid window. Any start discarded earlier already had a valid, shorter-or-equal candidate at an earlier right endpoint; it cannot improve the optimum later.

Trace the sample through right=4: window [2,3,1,2,4] may be shrunk repeatedly. Predict every length considered. Then implement both languages and test [], [8] target 7, [1,1] target 3, [1,1,1] target 2, and the sample.

Tempting wrong approach: replace while with if. It may leave a valid window unshrunk and miss the shortest answer. Second trap: apply this algorithm to [1,-1,5] with target 5. Trace it; why can removing the left value first make the sum drop below target even though another removal would increase it?

Exit ticket

State a recursive contract and a loop invariant, and explain their different roles. Derive total window time by counting pointer advances. Submit C/Python implementations, one merge trace, one partition trace, recurrences, and the negative-value counterexample. Use the appendix to correct the first failing case within today's time.

Solution appendix

Trace and proof checks

Merging [2,4,5] with [1,2,3] emits 1, left 2, right 2, 3, 4, 5. Taking from the left first at equal values preserves stability. Once one run is empty, choose from the other without indexing the exhausted run. Short-circuit guards in the reference implement this rule.

Partitioning [4,2,5,2,3] with pivot 3 first moves 2 ahead of 4, giving [2,4,5,2,3]; the second 2 then gives [2,2,5,4,3]. The final pivot swap produces [2,2,3,4,5], $p=2$. The two sides happen to be sorted here, but partitioning never promises that in general.

For $n \geq 2$, merge $T(n) = T(\text{floor}(n/2)) + T(\text{ceil}(n/2)) + \Theta(n)$. It is $\Theta(n \log n)$ in best, average, and worst cases for this version, with $O(n)$ buffer plus $O(\log n)$ stack, dominated by $O(n)$. Quicksort balanced $T(n) = 2T(n/2) + \Theta(n)$ is $\Theta(n \log n)$; $T(n) = T(n-1) + \Theta(n)$ is $\Theta(n^2)$. With uniformly random distinct-key permutations, average time is $\Theta(n \log n)$. Randomizing the pivot gives that expected time for distinct keys on any fixed input; equal-key degeneration remains for this partition rule. Stack depth is $O(\log n)$ balanced and $O(n)$ worst case.

Window checks

At $\text{right}=3$, sample sum is 8: record length 4 and remove 2, leaving 6. At $\text{right}=4$, sum becomes 10: record length 4, remove 3 (sum 7), record length 3, remove 1 (sum 6). At $\text{right}=5$, sum becomes 9: record length 3, remove 2 (sum 7), record length 2, remove 4 (sum 3). Answer: 2.

Each index is added once and removed at most once. Total time is $O(n)$, not $O(n^2)$, and workspace $O(1)$. This implementation scans all right endpoints, so for nonempty valid input its time is $\Theta(n)$.

On [1,-1,5], target 5, the algorithm records length 3, removes 1, gets sum 4, and stops shrinking. It returns 3 although [5] has length 1. Positivity supplied the monotonicity; without it, the inference that further removals cannot restore validity is false. The fixed-window algorithm from Day 2 does not have this restriction.

Progressive hints

Merge: write two endpoint guards before the comparison. Quicksort: draw all four regions, including the untouched pivot. Window: update the best length BEFORE removing the left value, and distinguish exact-sum bookkeeping from the positivity-based correctness proof.

Python 3 reference functions

PY

```
def merge_sort(a):
    buffer = [0] * len(a)

    def visit(lo, hi):
        if hi - lo < 2:
            return
        mid = lo + (hi - lo) // 2
        visit(lo, mid)
        visit(mid, hi)
        i, j = lo, mid
        for k in range(lo, hi):
            if i < mid and (j == hi or a[i] <= a[j]):
                buffer[k] = a[i]
                i += 1
            else:
                buffer[k] = a[j]
                j += 1
        for k in range(lo, hi):
            a[k] = buffer[k]

    visit(0, len(a))
```

PY

```
def partition(a, lo, hi):
    # Inclusive hi; caller supplies a nonempty valid range.
    pivot = a[hi]
    i = lo
    for j in range(lo, hi):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i
```

PY

```
def quick_range(a, lo, hi):
    if lo >= hi:
        return
    p = partition(a, lo, hi)
    quick_range(a, lo, p - 1)
    quick_range(a, p + 1, hi)
```

PY

```
def quick_sort(a):
    quick_range(a, 0, len(a) - 1)
```

PY

```
def min_window(a, target):
    # Contract: target > 0 and every value > 0.
    left = total = 0
    best = len(a) + 1
    for right, value in enumerate(a):
        total += value
        while total >= target:
            best = min(best, right - left + 1)
            total -= a[left]
            left += 1
    return 0 if best == len(a) + 1 else best
```

C11 reference functions

C

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>
```

C

```
static void merge_visit(int *a, int *buffer, size_t lo, size_t hi) {
    if (hi - lo < 2) return;
    size_t mid = lo + (hi - lo) / 2;
    merge_visit(a, buffer, lo, mid);
    merge_visit(a, buffer, mid, hi);
    size_t i = lo, j = mid;
    for (size_t k = lo; k < hi; ++k) {
        if (i < mid && (j == hi || a[i] <= a[j]))
            buffer[k] = a[i++];
        else buffer[k] = a[j++];
    }
    for (size_t k = lo; k < hi; ++k) a[k] = buffer[k];
}
```

C

```
bool merge_sort(int *a, size_t n) {
    if (n < 2) return true;
    if (n > SIZE_MAX / sizeof *a) return false;
    int *buffer = malloc(n * sizeof *buffer);
    if (!buffer) return false;
    merge_visit(a, buffer, 0, n);
    free(buffer);
    return true;
}
```

C

```
size_t partition(int *a, size_t lo, size_t hi) {
    int pivot = a[hi];
    size_t i = lo;
    for (size_t j = lo; j < hi; ++j) {
        if (a[j] <= pivot) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            ++i;
        }
    }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;
    return i;
}
```

C

```
static void quick_range(int *a, size_t lo, size_t hi) {
    if (lo >= hi) return;
    size_t p = partition(a, lo, hi);
    if (p > lo) quick_range(a, lo, p - 1);
    if (p < hi) quick_range(a, p + 1, hi);
}
```

C

```
void quick_sort(int *a, size_t n) {
    if (n > 1) quick_range(a, 0, n - 1);
}
```

C

```
size_t min_window(const int *a, size_t n, long long target) {
    /* Contract: positive target/values; exercise n <= 1000000. */
    size_t left = 0, best = n + 1;
    long long total = 0;
    for (size_t right = 0; right < n; ++right) {
        total += a[right];
        while (total >= target) {
            size_t length = right - left + 1;
            if (length < best) best = length;
            total -= a[left++];
        }
    }
    return best == n + 1 ? 0 : best;
}
```